Basic Maths Operator

L08 - All Maths Operators

Department of Computer Science
University of Pretoria

07 March 2024

Overview

1. Mathematical Operators
2. Operator Precedence
3. Comparison Operators
   - Boolean Expressions
4. Code example section
5. Input

Mathematical operators are classified as either unary or binary. The addition (+) and subtraction (-) operators are both unary and binary, while the other operators are all binary.

Mathematical operators in C++ are symbols or tokens that perform arithmetic operations on operands (values or variables).

These operators enable you to manipulate numeric data, perform calculations, and solve mathematical problems within your C++ programs.

Mathematical operators in C++:

- **Addition (+)**: Determines the sum of two operands.

```
int result = 5 + 3;  // result
```

- **Subtraction (-)**: Calculates the difference of two operands by subtracting the second operand from the first operand.

```
int result = 10 - 5;  //result
```

- **Multiplication (*)**: Determines the product of the two operands.

```
int result = 4 * 3;  // result
```

- **Division (/)**: Divides the left operand by the right operand. If both operands are integers, integer division is performed, and any fractional part is truncated. If one if the operands is a floating point value, floating point

division is performed and the result is a floating point value.

```
double result_double = 10.0 / 3
// result is 3.33333
double result_int = 10 / 3;
// result is 3
```

- **Modulus (%)**: Determines the remainder when dividing the left operand by the right operand.

```
int result=10 % 3;//result is 1
```

- **Increment (++) and Decrement** (−−): ++ increases the value of its operand by 1, while −− decreases the value of its operand by 1. There are prefix and
postfix versions of these operands as well.

If you use the ++ operator as a prefix like: ++var, the value of var is incremented by 1; then it returns the value.

If you use the ++ operator as a postfix like: var++, the original value of var is returned first; then var

is incremented by 1.

The -- operator works in a similar way to the ++ operator except -- decreases the value by 1.

- **Unary Plus (+) and Unary Minus (-)**: The unary plus operator (+) indicates the positive value of its

operand, while the unary minus operator (-) indicates the negative value of its operand.

```
int positive = +5;
//positive is 5

int negative = -5;
//negative is -5
```

These mathematical operators can be combined and used in expressions to perform complex calculations.

Understanding how to use these operators effectively is essential for solving mathematical problems and performing numerical computations in C++ programs.

Operator precedence in C++ determines:

1. The order in which operators are evaluated in an expression.
2. It defines which operators are evaluated first
3. and which are evaluated later when an expression contains multiple operators.

All mathematical expressions are evaluated according to the precedence of the operator involved. For example the expression:
$2 + 10 * 2$
will result in the answer of 22 and not 24. This is because multiplication (*) has higher precedence than addition (+).

Brackets can change the order in which expressions are evaluated.
For example, adding brackets to the expression given above as follows:
(2 + 10) * 2
will ensure that 2+10 is worked out first before the answer is multiplied by 2, this will give a final answer of 24.

The following list shows the precedence of the operators.
/, *, %, +,−

Note: /,* and % have higher priority than + and −.

The priority on a given level is equal,
that is + and − have the same priority.

This means that if an expression only has operators of the same precedence, the expression will be evaluated from left to right,

For example:
3 * 6 / 2

will result in 9 (3*6 = 18, 18 / 2 = 9 from left to right).

These mathematical operators can be combined and used in expressions to perform complex calculations.

Understanding how to use these operators effectively is essential for solving mathematical problems and performing numerical computations in C++ programs.

Compound Assignment Operators Compound assignment operators provide a shorthand way to update the value of a variable based on its current value and another value.

Since the variable's own value is used in the assignment it is important that the variable being updated already has a value.

Examples:

- `+=` (addition assignment): `a += b` is equivalent to `a = a + b`
- `-=` (subtraction assignment): `a -= b` is equivalent to `a = a - b`

There are compound assignment operators available for arithmetic,bitwise and shift

operations.

In C++, comparison operators are used to compare the values of two operands and determine their relationship. These operators return a boolean value (`true` or `false`) based on whether the comparison is true or false. Here's an explanation of the comparison operators in C++ along with examples:

- **Equal to (==):** This operator checks if the values of two operands are equal. If the values are equal, it returns `true`; otherwise, it returns `false`.

```
int a = 5;
int b = 7;

bool result = (a == b);
// result is false
```

- **Not equal to (!=):** This operator checks if the values of two operands are not equal. If the values are not equal, it returns `true`; otherwise, it returns `false`.

```
int x = 10;
int y = 10;
bool result = (x != y);
//result false
```

- **Greater than (>):** This operator checks if the value of the left operand is greater than the value of the right operand. If the condition is true, it returns `true`; otherwise, it returns `false`.

```
int p = 15;
int q = 10;
bool result = (p > q);//true
```

- Less than ($<$): This operator checks if the value of the left operand is less than the value of the right operand. If the condition is true, it returns true; otherwise, it returns false.

```
int m = 7;
int n = 12;
//result is true
bool result = (m < n);
```

- Greater than or equal to (>=): This operator checks if the value of the left operand is greater than or equal to the value of the right operand. If the condition is true, it returns true; otherwise, it returns false.

```
int x = 20;
int y = 20;
bool result = (x >= y);// true
```

- Less than or equal to (<=): This operator checks if the value of the left operand is less than or equal to the value of the right operand. If the condition is true, it returns true; otherwise, it returns false.

```
int a = 8;
int b = 10;
bool result = (a <= b);//true
```

These comparison operators are commonly used in conditional statements, such as `if` statements and loops, to control the flow of program execution based on the evaluation of conditions.

Understanding and mastering these operators is essential for writing effective and efficient C++ code.

A boolean expression (condition) is any number of expressions that either results in a `true` or `false`. For example, the expression that consists of a single condition, $2 > 3$, results in `false`. The following C++ condition will either in `true` or `false`, depending on the value of the variable used in the expression:

$(SavingsAccountBalance > 1000000.00)$

In C++, the boolean value `true` is represented by the integer value 1.

This convention allows boolean expressions to be evaluated in contexts where integer values are expected, such as in conditional statements (‘if‘, ‘while‘, etc.) and logical operators (&&, ||, etc.).

When a boolean expression evaluates to `true`, it is internally represented as the integer value 1.
Similarly, the boolean value `false` is represented by the integer value 0.
This allows for seamless integration of boolean logic with arithmetic and bitwise (& and |) operations in C++.

To place values into variables, as described in LO8, does not solve the problem of accepting different values user input into variables.
The function `cin` is used to read values from the console (or the keyboard).
The operation >> is defined to stream input from the keyboard into the variables
. `cin` is defined in the library `iostream` and

therefore, as with `cout`, it is necessary to specify the library using the preprocessor directive.

Listing 1 is an example program that reads an integer value from the keyboard and then uses this value in the subsequent output statement. Note the use of scoping operator to gain access to the `std` namespace.

Listing 1: Receiving input

```
#include <iostream>
using namespace std;
int main() {
    int x;

    cout << "Type in a value: ";
    cin >> x;
```

```
        cout << "x squared is: "
             << x * x << std::endl;

        return 0;
}
```

What output do you expect from the program? Run the program. Is the output the same as you expected?

Type in the following program:

Listing 2: Typing in two values

```
#include <iostream>
using namespace std;

int main() {
    float x, y;

    cout << "Type in two values: "
    cin >> x >> y;
```

```
        cout << "x times y is: "
             << x * y <<endl;

        return 0;
}
```

If the values of 10.6 and 30.000524 are typed in for x and y respectively, what is the output of the program?

If the variables being read in Listing 2 with the `cin` statement where read in using two statements, that is:

```
std::cin >> x;
std::cin >> y;
```

would the manner in which the values are read in change?

Run the program, as changed in Question 5, and type two values in with a space between

the values before pressing [Enter]. For example, type in:

```
10  50.3
```

and then press [Enter].

Variables may also be used to read in values of type string. To be able to manipulate strings in your program, you need to include the `string` library.

Consider the following program.

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string surname;
    char initial;

    cout<< "Type your surname: ";
```

```
    cin>> surname;

    cout << "Type your initial: ";
    cin >> initial;

    cout << "Hello " << initial
   << " " << surname <<endl;
    return 0;
}
```

When you run the program and enter "Sithole" for the surname and 'V' as the initial, the program will print: `Hello V Sithole`. What happens if the surname is "Van der Merwe"? The output is:

```
Type in your surname: Van der Merwe
Type in one of your first initial: Hel
```

The program took "Van" as the surname

that was typed in and ‘d’ as the initial. To fix this you need to use a special function, `getline` that does not use space as a delimiter.

Replace std :: cin >> surname;, with std :: getline (std :: cin , surname); and rerun the program. Test the program with a surname of ‘‘`Van der Merwe`’’ and inital of ‘V’. Is the output what you expected?

Below is the updated program.

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string surname;
    char initial;
```

```
cout << "Type your surname: ";
getline(cin, surname);

cout << "Type your initial: ";
cin >> initial;

cout << "Hello " << initial
<< " " << surname <<endl;
return 0;}
```

BODMAS rule

BODMAS is an acronym that stands for "Brackets, Orders (exponents and roots), Division and Multiplication (from left to right), Addition and Subtraction (from left to right)". It's a rule used to determine the order of operations when evaluating mathematical expressions.

Here's a breakdown of each part of the BODMAS rule:

1. Brackets: Operations within brackets are performed first. If there are nested brackets, the innermost brackets are evaluated first.

2. Orders (Exponents and Roots): After dealing with brackets, operations involving exponents (powers) or roots (such as square roots) are performed.

3. Division and Multiplication (from left to right): After brackets and exponentiation, division and multiplication operations are performed from left to right.

4. Addition and Subtraction (from left to right): Finally, addition and subtraction operations are performed from left to right.

Following the BODMAS rule ensures that mathematical expressions are evaluated correctly and consistently. Here's a simple example to illustrate how BODMAS works:

Consider the expression: 4+(3 − 1)∗2
1. Brackets: First, we evaluate the expression within the brackets: 3 − 1 = 2
2. Multiplication: Next, we perform the multiplication: 2 ∗ 2 = 4
3. Addition: Finally, we add the result to 4: 4 + 4 = 8

So, the value of the expression 4+(3 − 1)∗2 is 8

Without following the BODMAS rule, the result might be different.

Here are some examples to illustrate the difference between the / and (%) :

What is the Output of program 1 and program 2?

```
//Program 1
#include <iostream>
#include <string>
using namespace std;

int main() {
    // Input for four values
    double a = 1;
    double b = 2;
    double c = 3;
    double d = 4;

    // Applying BODMAS rules
    double result = (a + b) * (d - c) / (a * b);

    // Output the result
    cout << "Result of BODMAS operation: " << result << endl;
    return 0;
```

Let's break down the expression step by step following the BODMAS rules:

1. Brackets: There are no brackets in this expression, so we move on to the next step.

2. Orders (Exponents and Roots): There are no exponents or roots in this expression, so we move on.

3. Division and Multiplication (from left to right):
We first perform addition: a + b = 1 + 2 = 3.
We then perform subtraction: d − c = 4 − 3 = 1.
Finally, we perform multiplication: a * b = 1 * 2 = 2.

4. Division: After completing all multiplication and division operations, we perform division: (a+b)*(d−c)/(a*b)=3*1/2=1.5.

```
//Program 2
#include <iostream>
#include <string>
using namespace std;

int main() {
    // Input for four values
    int a = 1;
    int b = 2;
    int c = 3;
    int d = 4;

    // Applying BODMAS rules
    double result = (a + b) * (d - c) % (a * b);

    // Output the result
    cout << "Result of BODMAS operation: " << result << endl;
    return 0;
```

Let's break down the expression step by step following the BODMAS rules:

1. Brackets: There are no brackets in this expression, so we move on to the next step.

2. Orders (Exponents and Roots): There are no exponents or roots in this expression, so we move on.

3. Division and Multiplication (from left to right):
We first perform addition: a + b = 1 + 2 = 3.
We then perform subtraction: d − c = 4 − 3 = 1.
Next, we perform multiplication: a * b = 1 * 2 = 2.

Finally, we perform modulo: (a+b)*(d−c)%(a*b) = 3*1%2.

The modulo operation returns the remainder of the division operation, so 3*1%2 = 3%2 = 1.

modulo gives the remainder of the devision of 3 by 2

```
//What is the output of this program?
#include <iostream>
#include <string>
using namespace std;

int main() {
    // Increment and decrement operators
    int num1 = 10;
    cout<< "\n \nStarting output of num1"<<endl;
    cout << "Initial value of num: " << num1 << endl;
    cout << "Incrementing num: " << ++num1 << endl;
    num1 = 10;
    cout << "Incrementing num: " << num1++ << endl<< endl;

    int num2 = 10;
    cout<< "Starting output of num2"<<endl;
    cout << "Initial value of num: " << num2 << endl;
    cout << "Decrementing num: " << --num2 << endl;
    num2 = 10;
    cout << "Decrementing num: " << num2-- << endl<< endl;
    return 0;
}
```

```
Starting output of num1
Initial value of num: 10
Incrementing num: 11
Incrementing num: 10

Starting output of num2
Initial value of num: 10
Decrementing num: 9
Incrementing num: 10
//////////////////////////
explanation:
++ and -- operator as prefix and postfix
If you use the ++ operator as a prefix like: ++var,
the value of var is incremented by 1; then it returns the value.
If you use the ++ operator as a postfix like: var++,
the original value of var is returned first;
then var is incremented by 1.

The -- operator works in a similar way to the ++
operator except -- decreases the value by 1.
```

It is important to note that:

- When reading in values from the keyboard, one should always have accompanying explanatory cout statements before the cin statements.

  Then the user of the program will know what to enter, for example:

```
cout << "Enter two numbers separated by a space:");
```

- When we discuss pointers and arrays, we will revisit string input.